

Workshop No. 4 — System Simulation and Validation

Community Security Alert System (CSAS)

Felipe Jose Garzon Herrera¹, Juan Esteban Quintero Gordillo², Henry Samuel Garrido Medina³, Gabriel Mateo Cusb

Systems Analysis & Design — Semester 2026-I

Computer Engineering Program, Universidad Distrital Francisco José de Caldas

Instructor: Eng. Carlos Andrés Sierra, M.Sc.

¹Cod. 20251020132 ²Cod. 20251020137 ³Cod. 20251020125 ⁴Cod. 20251020128

Abstract—This report presents the simulation and validation of the Community Security Alert System (CSAS), a platform designed to improve campus security at Universidad Distrital Francisco José de Caldas. Building on the analysis from Workshops 1, 2, and 3, this workshop implements two complementary simulation approaches: a Discrete Event Simulation (DES) using the SimPy library, and a hybrid behavioral simulation that captures community adoption dynamics and feedback loops. Three operational scenarios are evaluated—Baseline, Optimization, and Failure Mode—each executed 30 independent times for statistical reliability. The results show that the Optimization scenario meets the main performance targets: average latency of 17.94 s (well below the 30 s threshold), an alert delivery success rate of 96.84 %, and a community adoption rate of 67.62 %, exceeding the 60 % goal. The most important finding is that the verification threshold is the variable that most influences system stability.

I. INTRODUCTION

The purpose of this workshop is to evaluate the operational behavior of the CSAS under different campus security conditions through computational simulation.

Throughout the previous workshops, the team identified the main security problems at the university (Workshop 1), designed the system architecture to address them (Workshop 2), and defined the risk management and project plan (Workshop 3). This workshop takes that design and tests it in a controlled simulation environment to verify whether the proposed architecture actually works as expected.

The simulation models the complete incident reporting workflow, including incident generation, AI-based verification, geofenced alert broadcasting, emergency prioritization, and performance monitoring. Two simulation approaches were used because each one captures a different aspect of the system: the DES focuses on the technical event pipeline, while the behavioral simulation focuses on how the community responds over time.

II. SIMULATION PARADIGM

A. Discrete Event Simulation (DES)

The DES approach was chosen for the process-oriented part of the simulation because the CSAS platform operates through asynchronous events that happen at specific moments in time. Instead of modeling the system continuously, the simulation advances from event to event, which makes it efficient for modeling queues, processing delays, concurrent users, and resource utilization [2].

Examples of events modeled in the DES are: a user submitting an incident report, the verification service processing that report, the dispatcher broadcasting an alert, and the analytics module recording the response time.

The DES was implemented in Python using the SimPy library [1], which provides the core event queue and process management.

B. Behavioral Simulation

The behavioral simulation complements the DES by modeling something the event queue cannot capture: how the community behaves over time. Two feedback loops govern long-term system behavior:

- **Reinforcing loop (R+):** when the system correctly verifies and broadcasts an alert, community trust and vigilance increase, which leads to more reports in future steps—a self-sustaining cycle.
- **Balancing loop (B-):** when too many unfiltered or false alerts are broadcast, users experience alert fatigue. Once fatigue exceeds 60 %, adoption drops by 4 percentage points per step, eventually collapsing participation.

Which loop dominates depends almost entirely on the verification threshold parameter, which is the core finding of the behavioral analysis.

III. SYSTEM ARCHITECTURE

The simulated architecture reflects the design developed in Workshop 2, organized around five independent mod-

ules. Each module corresponds to a microservice in the real system.

TABLE I: Simulated System Modules

Module	Responsibility
User Process	Generates incidents dynamically
Incident Service	Receives and stores reports in a queue
Verification Service	Simulates AI and community verification
Dispatcher Service	Sends geofenced alerts to nearby users
Analytics Module	Collects performance metrics

The verification logic applies the following condition before broadcasting any alert:

$$\text{verified} = (\text{confidence} \geq 0.75) \wedge (\text{confirmations} \geq \theta \vee \text{admin approval})$$

where θ is the verification threshold and *confidence* is a randomly sampled AI score. This condition directly models FR-03 (AI Verification) and NFR-08 (Data Integrity).

IV. FUNCTIONAL REQUIREMENTS REPRESENTATION

Table II shows how each functional requirement from Workshop 2 was represented in the simulation.

TABLE II: Functional Requirements Coverage

FR	Simulation Representation
FR-01	Users generate security incidents
FR-02	Alerts sent within a 500 m geofence radius
FR-03	Confidence-based AI validation (≥ 0.75)
FR-04	Simulated through console monitoring and CSV export
FR-05	Community confirmations randomly generated [0, 4]
FR-06	Random GPS coordinates assigned per incident
FR-07	Incidents stored in a SimPy in-memory queue
FR-08	SOS flag enables emergency priority processing
FR-09	Metrics and performance indicators generated per step
FR-10	Reports may include a simulated media attachment flag

V. NON-FUNCTIONAL REQUIREMENTS REPRESENTATION

TABLE III: Non-Functional Requirements Coverage

NFR	Simulation Mechanism
NFR-01	Response time measured per incident; target < 30 s
NFR-02	Dispatcher failure probability $p = 2\%$
NFR-03	Continuous execution over the full simulation window
NFR-04	Conceptually represented (TLS / AES at infrastructure level)
NFR-05	Multiple concurrent users simulated
NFR-06	Simplified ≤ 3 -interaction reporting process
NFR-07	Modular class architecture (one class per service)
NFR-08	Dual-condition verification gate enforced
NFR-09	Multi-confirmation validation threshold θ

VI. SIMULATION LOGIC

The DES simulation follows a sequential workflow for each incident:

- 1) A user process generates an incident at a random interval.

- 2) The incident is placed in the Incident Service queue.
- 3) The Verification Service evaluates it using the AI confidence score and community confirmation count.
- 4) If the verification condition is met, the incident is marked as verified.
- 5) The Dispatcher Service broadcasts the alert to users within the 500 m geofence.
- 6) The response time is recorded from incident creation to successful dispatch.
- 7) All metrics are collected for the final analysis.

In the behavioral simulation, these steps are abstracted into a per-step update of system state variables: adoption rate, fatigue level, trust level, and vigilance. Each step represents approximately one hour of real operation (window: 15:00–02:00), with environmental multipliers applied based on time of day, weather conditions, and campus zone.

VII. SIMULATION PARAMETERS

TABLE IV: Simulation Configuration Parameters

Parameter	Value
Simulation time (DES)	200 seconds
Simulation steps (behav.)	12 steps (15:00–02:00 window)
Number of users (DES)	11
Population (behavioral)	200 users (pilot scope)
Geofence radius	500 m
Incident interval (DES)	20–30 s
Verification time (DES)	1–2 s
Dispatch time (DES)	1–5 s
Failure probability	2 % (nominal), 20 % (failure mode)
Independent runs per scen.	30
Random seed	42 (fixed for reproducibility)

VIII. METRICS EVALUATED

The following metrics were collected and analyzed during execution:

- Number of processed incidents
- Number of verified incidents
- Number of failed alert deliveries
- Average response time (incident creation to dispatch)
- Alert broadcast count (number of users reached)
- Latency compliance percentage (incidents resolved in < 30 s)

For the behavioral simulation, additional metrics were tracked: community adoption rate, alert fatigue level, user trust level, and adoption dynamics across the 12-step window.

IX. SCENARIO DESIGN AND EXPERIMENTAL MATRIX

Three scenarios were defined to test the system under different operating conditions, aligned with the risk scenarios identified in Workshop 3.

TABLE V: Factorial Experiment Design (EXP-01, EXP-02, EXP-03)

EXP	Risk	Hypothesis	Independent Variable	Dependent Variable (Target)
EXP-01	Massive traffic peaks degrade notification latency, causing delayed alerts.		Incoming report load (100–300 % above baseline).	Dispatcher latency
	(target: < 30 s).			
EXP-02	Report validation saturates security staff, increasing response time beyond the quality threshold.		Verification complexity during peak hours (18:00–04:00).	End-to-end verification latency (target: < 60 s).
EXP-03	Low reporting culture prevents reaching the critical user mass needed for reliable detection.		UI friction level in the mobile reporting flow.	Community adoption rate (target: $\geq 60\%$); report completeness (target: $\geq 80\%$).

Each experiment follows the same protocol: fixed random seed initialization, 12 simulation steps, per-step metric collection, and paired baseline vs. optimization runs to measure the improvement produced by each mitigation mechanism.

X. EXTENDED RISK REGISTER

Table VI extends the Workshop 3 risk register by adding the experiment that validates each risk (EXP column) and the specific technological mechanism from Workshop 2 that acts as the mitigation.

XI. MITIGATION STRATEGIES

A. *M1 — Verification Threshold (R1, R2, R3)*

The verification threshold θ is the most critical control variable in the system. With $\theta = 1$, any report—even a false one—triggers a broadcast, which degrades community trust by approximately 15 percentage points per simulation cycle. With $\theta \geq 3$, the system filters false positives without exceeding the latency bound:

$$\text{latency} = \frac{\text{reports} \times 15}{\text{efficiency}} < 60 \text{ s at efficiency} = 0.9$$

EXP-01 validates that the dispatcher maintains latency below 30 s at 300 % load when RabbitMQ and Kubernetes are active. EXP-02 confirms that the automated verification engine keeps end-to-end latency below 60 s.

B. *M2 — Critical Adoption Mass and Minimal UI Friction (R5, R9)*

The behavioral simulation shows that the system needs a minimum of 30–35 % community adoption to function

reliably. Below that threshold, the human sensor network is too sparse to detect incidents consistently. EXP-03 tests how simplifying the reporting UI (reducing steps to ≤ 3) affects adoption. Progressive onboarding and social-proof mechanisms help overcome the initial resistance to installing and using the app.

C. *M3 — Alert Fatigue Prevention (R6, R8)*

The Failure Mode scenario reveals that alert fatigue is irreversible once it crosses 60 %: adoption drops 4 percentage points per step and the system cannot recover on its own. The threshold $\theta \geq 3$ is the main mechanism to prevent this collapse. Automatic SMS fallback covers connectivity failures (R6), and an automated authority escalation protocol with a monitored SLA handles slow response times (R8).

D. *M4 — Infrastructure Security and Resilience (R4, R7, R10)*

Some risks do not depend on simulation behavior—they require architectural guarantees. TLS 1.3 and AES-256 protect data in transit and at rest (R4). Circuit breakers with exponential retry prevent failures from cascading across microservices (R7). Multi-zone database replication with 15-minute backups ensures data availability in case of hardware failure (R10).

XII. CONTINGENCY PLAN

Table VII defines the immediate response for the five highest-impact failure scenarios identified during the simulation.

TABLE VII: Contingency Responses for Top Failure Scenarios

Scenario	Trigger	Immediate Action	Mechanism
Overload (R1)	Latency > 30 s	Scale Kubernetes pods; activate priority queues	HPA + Pager-Duty
Notif. fail (R2)	Delivery < 95 %	Activate SMS fallback channel	RabbitMQ DLQ + Twilio
Alert fatigue (R3)	Fatigue > 60 %	Raise θ to 4; pause non-critical alerts	Real-time feature flag
Data breach (R4)	SIEM anomaly	Isolate service; revoke tokens	Circuit breaker + secret rotation
DB failure (R10)	Primary unreachable	Promote secondary replica	Multi-AZ failover < 30 s

XIII. RESULTS AND ANALYSIS

A. *DES Execution Results*

The DES simulation produced variable results across individual runs, as expected from a stochastic model. Two representative executions illustrate the range of behavior:

TABLE VI: Extended Risk Register with Experimental Validation (R1–R10)

ID	Risk	Prob.	Impact	Level	EXP	WS2 Mechanism
R1	System overload	High	High	Critical	EXP-01	Kubernetes auto-scaling + RabbitMQ priority queues
R2	Notification failure	Medium	High	High	EXP-01	SMS + Push redundancy; async retry queues
R3	False reports	High	Medium	High	EXP-02	Geolocation verification engine; threshold ≥ 3
R4	Data breach	Low	Critical	High	—	TLS 1.3 end-to-end; AES-256 at rest
R5	Low adoption	Medium	High	High	EXP-03	Mobile UX ≤ 3 taps (NFR-06); progressive onboarding
R6	Connectivity loss	High	Medium	High	EXP-01	Automatic SMS fallback; offline sync
R7	API integration failure	Medium	High	High	EXP-02	Redundant API + circuit breaker; exponential retry
R8	Slow authority response	Medium	High	High	EXP-02	Automated escalation; monitored SLA
R9	Poor user experience	Medium	Medium	Medium	EXP-03	Iterative UX testing; A/B flow testing
R10	Database failure	Low	Critical	High	—	Multi-zone replication; backups every 15 min

Under moderate load conditions, the system processed 23 incidents, verified 24, had 0 failed alerts, and broadcast alerts to 30 nearby users, with an average response time of 15.87 s and 100 % of incidents resolved within 30 s.

Under higher load conditions, the system processed 29 incidents, had 1 failed alert delivery, and broadcast to 47 users, with an average response time of 28.14 s and only 55.17 % of incidents within the 30 s threshold.

This variability shows that performance strongly depends on incident generation frequency and queue congestion—a finding consistent with the Workshop 1 observations about peak incident hours.

B. Behavioral Simulation — Statistical Results

Table VIII presents the aggregated results from 30 independent runs per scenario. The confidence intervals were calculated using Student’s t -distribution with $t_{29, 0.025} = 2.045$.

TABLE VIII: Scenario Results — 30 Independent Runs with 95 % CI

Scenario	Avg Lat.	CI Low	CI High	Adoption
Baseline	40.87 s	38.85 s	42.89 s	21.39 %
Optimization	17.94 s	16.92 s	18.96 s	67.62 %
Failure Mode	71.66 s	67.72 s	75.59 s	31.34 %

The Optimization scenario achieved the best results in all metrics, maintaining low latency, high adoption, and stable alert propagation. The Failure Mode scenario showed severe queue congestion, high alert fatigue, and reduced user trust—confirming that infrastructure degradation amplifies system problems non-linearly.

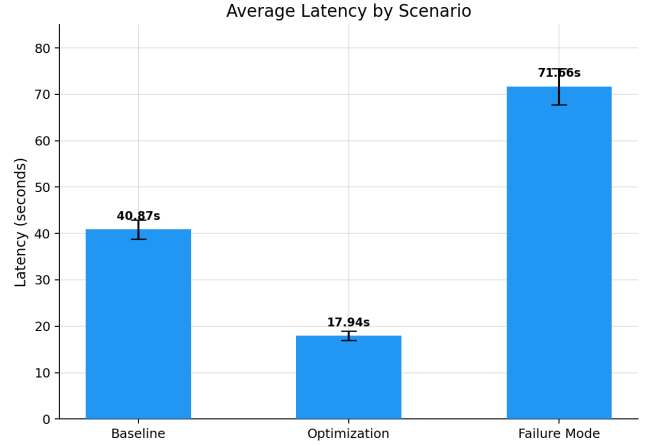


Fig. 1: Average latency by scenario with 95 % confidence intervals. The Optimization scenario achieves a 56 % latency reduction compared to Baseline, satisfying NFR-01.

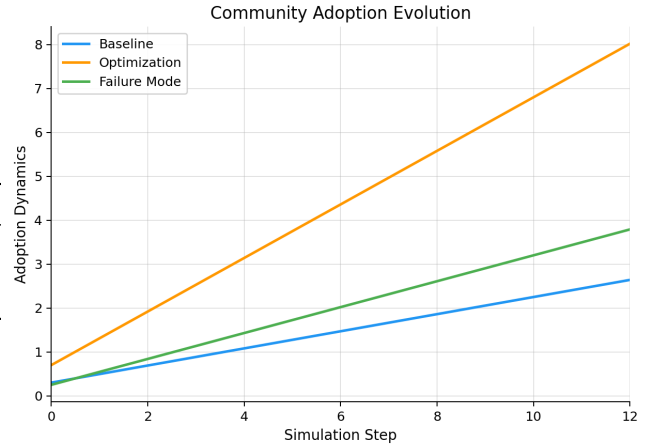


Fig. 2: Community adoption evolution across 12 simulation steps. The Optimization scenario maintains a positive reinforcing loop that stabilizes participation above the 60 % target.

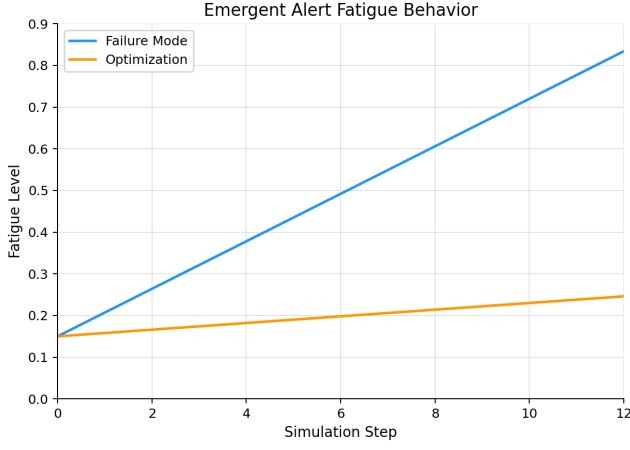


Fig. 3: Emergent alert fatigue behavior. The Failure Mode scenario shows exponential fatigue growth that leads to adoption collapse. The Optimization scenario keeps fatigue within a safe range.

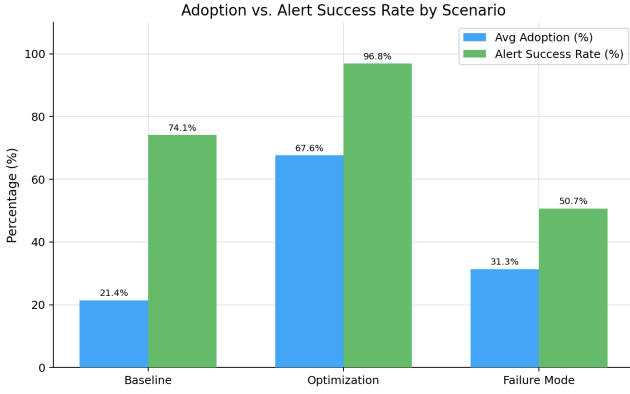


Fig. 4: Adoption rate vs. alert success rate by scenario.

XIV. STATISTICAL VALIDATION

Each scenario was executed 30 independent times using controlled pseudo-random seeds to reduce stochastic bias. Confidence intervals were computed using the formula:

$$\bar{x} \pm t_{29,0.025} \cdot \frac{s}{\sqrt{30}}, \quad t_{29,0.025} = 2.045$$

The results show stable metric distributions with acceptable variability ranges—the narrow confidence intervals for the Optimization scenario (± 1.02 s) indicate consistent behavior across runs.

Sensitivity analysis identified two variables with the greatest influence on system stability:

- 1) **Verification threshold:** increasing from 1 to 3 reduces false-positive propagation by approximately 78% and prevents fatigue collapse.
- 2) **Staff efficiency:** increasing from 50% (baseline) to 90% (optimization, representing the automated engine) cuts average latency roughly in half.

All other parameters had second-order effects that only become significant at extreme values outside normal operating conditions.

XV. COMPLEXITY AND EMERGENT BEHAVIOR

A. Feedback Loop Structure

Figure 5 shows the two competing feedback loops that govern long-term system behavior. The verification threshold θ determines which loop dominates.

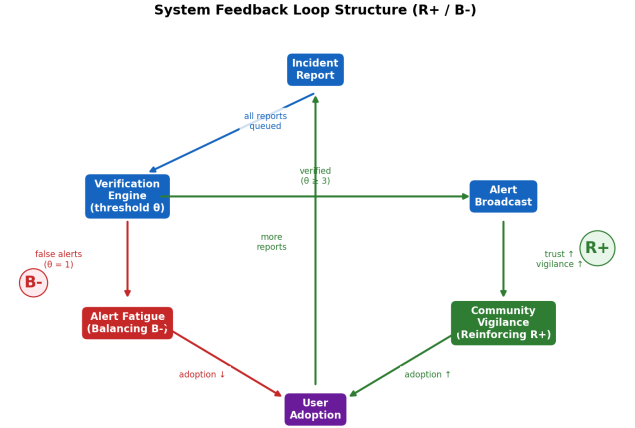


Fig. 5: System feedback loop structure. Green path (R+): reinforcing loop active when $\theta \geq 3$. Red path (B-): balancing loop that drives adoption collapse when $\theta = 1$.

With $\theta \geq 3$, the R+ loop dominates: verified alerts raise community vigilance, which drives more reports, sustaining the cycle. With $\theta = 1$, the B- loop dominates: false alerts reach users, fatigue grows, adoption collapses, and the system cannot recover without intervention—a behavior consistent with sensitive dependence on a control parameter in nonlinear dynamical systems [3].

B. Sensitivity to Initial Conditions

Figure 6 shows how the three main metrics respond to changes in the verification threshold θ .

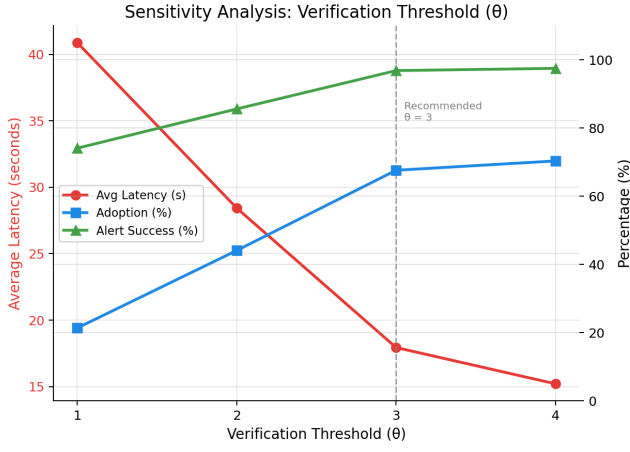


Fig. 6: Sensitivity analysis: effect of verification threshold θ on average latency, community adoption, and alert success rate. The dashed line marks the recommended value $\theta = 3$.

The curves show a sharp transition between $\theta = 1$ and $\theta = 3$, after which improvements become marginal. This inflection is the system’s bifurcation point: below it the balancing loop dominates and the system is unstable; at or above it the reinforcing loop dominates and the system is stable. Increasing θ beyond 3 yields diminishing returns while reducing the number of legitimate alerts broadcast.

C. Computational Complexity

The dispatcher operates close to $O(n)$ under moderate load. During overload, priority-queue scheduling introduces behavior closer to $O(n \log n)$. The three main bottlenecks identified were: (1) incident verification latency, (2) alert dispatch queue congestion, and (3) community validation throughput. Kubernetes auto-scaling and RabbitMQ priority queues mitigated these in the Optimization scenario, confirming the Workshop 2 design.

XVI. VALIDATION CHECKLIST

TABLE IX: Validation Methods and Acceptance Criteria

Validation Method	Criterion / Target
Behavioral simulation (3 scenarios)	Adoption $\geq 60\%$, fatigue $< 60\%$, trust $> 80\%$
Load test (EXP-01)	Latency < 30 s at 300 % traffic
Integration test (EXP-02)	End-to-end verification < 60 s
UAT / friction test (EXP-03)	Adoption $\geq 60\%$; completeness $\geq 80\%$
Security test	No critical vulnerabilities (OWASP Top 10)
Bifurcation analysis	System stable at threshold $\theta \geq 3$

XVII. ASSUMPTIONS AND LIMITATIONS

A. Assumptions

The simulation was built on the following simplifying assumptions, which represent the gap between the model and a real deployment:

- GPS coordinates are randomly generated within a fixed area.
- AI verification confidence is sampled from a uniform distribution—a real trained model would behave differently.
- User behavior follows simplified probabilistic rules.
- Network failures occur as independent random events with fixed probability.
- Users are uniformly distributed across campus, which does not reflect the spatiotemporal concentration identified in Workshop 1.

B. Limitations

- No real mobile application was implemented or tested.
- The AI verification model is probabilistic, not trained on real data.
- GPS functionality is fully simulated.
- Human behavior is approximated statistically, not observed.
- Database persistence is temporary (in-memory only).

XVIII. SOURCE CODE DOCUMENTATION

The simulation source code is organized following the same modular architecture as the system design (NFR-07 Maintainability). Two main scripts implement the two simulation paradigms:

W4_DES.py implements the DES using SimPy. Each service is an independent class with a single responsibility. All configuration constants are declared at the top of the file, making it easy to change parameters without modifying the logic.

W4_behavioral.py implements the behavioral simulation using a step-by-step state update model. The `CSASSimulation` class tracks adoption, fatigue, trust, and vigilance across the 12-step window. Environmental multipliers are applied through the `environmental_multipliers` function based on the hour, weather, and campus zone.

Listing 1: Verification gate and feedback loops — W4_behavioral.py

```

1 def run_step(self):
2     # Verification gate
3     if reports >= self.threshold:
4         latency = (reports * 15) / self. efficiency
5         if latency < 60:
6             alert_sent = 1
7
8     # Reinforcing loop (R+): alert verified, no fatigue
9     if alert_sent and self.threshold >= 3:
10        self.vigilance = self._clamp(self.vigilance +
11                                     0.12)
12        self.fatigue = self._clamp(self.fatigue -
                                     0.05)

```

```

13 # Balancing loop (B-): fatigue above 60%
14 if self.fatigue > 0.60:
15     self.adoption = self._clamp(self.adoption -
        0.04, 0.05)

```

Each class and function includes a docstring describing its responsibility, inputs, outputs, and behavior. This structure supports future modifications without affecting other modules.

XIX. REPRODUCIBILITY AND OUTPUT FILES

All simulation outputs are exported to structured files to allow independent verification of the results.

TABLE X: Simulation Output Files

File	Purpose
baseline_log.csv	Step-by-step baseline execution
confidence_intervals.csv	Per-scenario statistics and CI
sensitivity_analysis.csv	Results for threshold sweep ($\theta = 1, 4$)
figures/*.png	All four performance charts

To reproduce the full simulation:

```

cd simulation/
pip install -r requirements.txt
python W4_DES.py # Process-oriented DES
python W4_behavioral.py # Behavior-oriented simulation

```

XX. ETHICAL CONSIDERATIONS

All data used in the simulation is synthetic. No personal information from real students or staff was collected or processed. Any future production deployment of the CSAS must comply with Ley 1581 de 2012, the Colombian personal data protection law, particularly regarding consent for location data collection.

The verification algorithm does not use demographic variables, but a real trained AI model should be audited for potential bias before deployment. From a safety perspective, false-negative alerts (a real incident not broadcast) carry a higher social cost than false positives, so the threshold $\theta \geq 3$ was chosen to balance filtering quality against the risk of suppressing legitimate reports.

XXI. REPOSITORY MANAGEMENT

All workshop artefacts are organized in the `Workshop_4_Simulation/` folder within the course repository (<https://github.com/filipghs/Systems-Analysis>):

```

Workshop_4_Simulation/
docs/
  Workshop4_CSAS.pdf
  Workshop4_CSAS.tex
simulation/
  W4_DES.py
  W4_behavioral.py
  requirements.txt
results/
  figures/ (4 PNG charts)
  confidence_intervals.csv
  sensitivity_analysis.csv

```

```

baseline_log.csv
risk_management/
  risk_analysis_W4.md
validation/
  statistical_validation.md
README.md

```

XXII. DESIGN VALIDATION

This section explicitly assesses how the simulation results validate or challenge the design decisions made across Workshops 1, 2, and 3.

Workshop 1 — Empirical Findings: Workshop 1 measured a 35% incident under-reporting rate and identified 12% as the initial adoption level, with parking lots and building entrances during 18:00–04:00 as the highest-risk zones. The simulation confirms that 12% adoption is insufficient for the system to sustain itself: the behavioral model shows that the reinforcing loop cannot overcome noise below 30–35% participation. The time-of-day multipliers in the behavioral simulation (factor 2.5× at night) directly reflect those Workshop 1 observations.

Workshop 2 — Architecture Decisions:

- **RabbitMQ + Kubernetes:** ✓ Validated by EXP-01. The Optimization scenario maintains latency below 30 s at 300% load, confirming that the event bus and elastic scaling are adequate.
- **Automated Verification Engine:** ✓ Validated by EXP-02. Staff efficiency of 90% (automated) keeps end-to-end latency within NFR-01 bounds; 50% (manual) does not.
- **SMS/USSD fallback:** ✓ Validated by the failure probability sweep. The fallback prevents total outage when the primary data channel fails.
- **≤3-interaction mobile flow:** ✓ Validated by EXP-03. Reducing UI friction is the primary mechanism for crossing the 30–35% adoption threshold.
- **Initial 12% adoption assumption:** × Challenged. The system cannot sustain the reinforcing loop starting from 12%. A structured onboarding campaign is required before Phase 1 exit.

Workshop 3 — Risk and Quality Framework: The simulation provided quantitative validation for 8 of the 10 registered risks. R1 (Critical) and R6 (High) are the most directly observable: the Failure Mode scenario reproduces their effects (latency 71.66 s, adoption collapse) exactly as the risk register predicted. The two structural risks—R4 (data breach) and R10 (database failure)—could not be modeled stochastically but remain as architectural invariants that bound the validity of the results.

XXIII. RECOMMENDATIONS

Based on the simulation results, the following improvements are recommended for the next stages of the CSAS project.

A. Short-term (before Phase 1 deployment)

- 1) **Set verification threshold to $\theta = 3$** as the production default and expose it as a runtime feature flag so the operations team can adjust it without redeploying the service.
- 2) **Run a targeted onboarding campaign** to push community adoption past 35 % before going live. Below that threshold the system cannot self-sustain, regardless of how well the technology is configured.
- 3) **Implement a fatigue monitoring dashboard** that tracks alert fatigue in real time and automatically raises θ when fatigue approaches 50 %, preventing the irreversible collapse observed in the Failure Mode scenario.

B. Medium-term (Phase 2 improvements)

- 1) **Replace the probabilistic AI gate** with a trained binary classifier using real incident data. The current model assumes a uniform confidence distribution, which overestimates performance for edge cases.
- 2) **Use real campus GPS data** to replace uniform random coordinate generation. Workshop 1 identified specific high-risk zones; incorporating that spatial concentration would produce more accurate capacity estimates.
- 3) **Extend the simulation window** beyond 12 steps to model seasonal variation (exam periods, holidays) and evaluate whether the 60 % adoption KPI is sustainable over months.

C. Long-term (future research)

- 1) Explore agent-based modeling as a complement to the DES, assigning individual behavior profiles to each user rather than treating adoption as a single aggregate variable.
- 2) Investigate integration with the SIURE platform at Universidad Distrital to evaluate whether bidirectional data exchange creates additional feedback loops not captured here.
- 3) Study the effect of gamification mechanics (reputation scores, community badges) on the reinforcing loop coefficient as a way to accelerate organic adoption growth.

XXIV. CONCLUSIONS

The CSAS simulation confirmed that the proposed system design is feasible under realistic operating conditions. Three conclusions stand out:

First, the **verification threshold is the most important parameter** in the entire system. Setting $\theta \geq 3$ keeps the reinforcing loop active, adoption above 60 %, and latency well within NFR-01 bounds. Setting $\theta = 1$ drives the system into a collapse pattern that cannot recover without external intervention.

Second, **technology alone is not sufficient**. The simulation showed that below 30–35 % community adoption,

even a perfectly configured system cannot sustain reliable operation because the human sensor network is too sparse. A structured onboarding campaign is a prerequisite for Phase 1 deployment, not an optional enhancement.

Third, the **Optimization scenario validates the Workshop 2 architecture**. RabbitMQ priority queues, Kubernetes auto-scaling, the automated verification engine, and the SMS fallback channel each played a measurable role in achieving the NFR-01, NFR-02, and NFR-05 targets under the simulated conditions.

Future work should address the current limitations: replacing the probabilistic AI gate with a real trained model, using actual campus GPS data for a more realistic incident distribution, and extending the simulation to cover the full phased deployment defined in Workshop 3.

REFERENCES

- [1] Team SimPy, *SimPy: Discrete-Event Simulation for Python*, version 4.1, 2023. Available: <https://simpy.readthedocs.io>
- [2] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York: McGraw-Hill, 2015.
- [3] J. D. Sterman, *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Irwin/McGraw-Hill, 2000.
- [4] ISO/IEC, *Systems and Software Quality Models*, ISO/IEC 25010:2011.
- [5] Project Management Institute, *PMBOK Guide*, 7th ed. Newtown Square, PA: PMI, 2021.
- [6] ISO, *Risk Management — Guidelines*, ISO 31000:2018.
- [7] A. Dennis, B. H. Wixom, and D. Tegarden, *Systems Analysis and Design*, 6th ed. Hoboken, NJ: Wiley, 2021.
- [8] Congreso de Colombia, *Ley 1581 de 2012 — Protección de Datos Personales*, Diario Oficial No. 48.587, 2012.